

Analyse v13 → v15 — Cache RS statique + Phase 2 adaptative SEUIL_1NEWTON

Date : 2026-07-04

Branche : Riemann_Lab_C

Auteur : hprzeta

1. Problèmes de v13 (Illinois hybride 2-phases, référence)

1.1 Goulot Phase 2 — coût arb_fpwrap_cdouable_hardy_z

v13 atteint 8.50 min sur T=100 000 avec l'architecture 2-phases :

Phase	Coût	Part du total
Phase 1 — Illinois Z_rs_double	~0.36 ms/zéro (24 iter × 0.015 ms)	~2 %
Phase 2 — 2 Newton Z_arb	~3.6 ms × 2 = 7.2 ms/zéro	~96-98 %
Scan C — Z_double + changement de signe	~0.015 ms/point	< 1 %

Cause : arb_fpwrap_cdouable_hardy_z coûte ≈ 1.8 ms/appel à T=100k (N_RS ≈ 126 termes). Chaque zéro nécessite 2 appels par Newton step × 2 steps → **4 appels Z_arb = 7.2 ms** en Phase 2.

La Phase 1 est quasi-gratuite (0.015 ms/appel) mais la Phase 2 domine à ≈ 98 %.

1.2 Coût Z_rs_double : $\log(n) + 1/\sqrt{n}$ répétés à chaque évaluation

Pour chaque évaluation Z_rs_double, la boucle RS calcule :

```
/* v13 - coûteux */  
for (n = 1; n <= N; n++)  
    sum += cos(th - t * log((double)n)) / sqrt((double)n);
```

À chaque appel : $\log()$ ≈ 50 cycles + $\sqrt{()}$ ≈ 50 cycles = 100 cycles/terme. Pour N=126 termes (T=100k) : $\sim 12\,600$ cycles/évaluation.

Bien que Phase 1 ne représente que 2 % du total, ces calculs répétés s'accumulent sur les 24 itérations Phase 1 × 138 069 zéros = **3 313 656 évaluations Z_rs_double**.

1.3 Phase 2 : 2 Newton steps pour TOUT t, même à grand t

La Phase 2 de v13 applique systématiquement **2 Newton steps** quel que soit t :

```
/* v13 - 2 Newton fixes pour tout t */  
for (int k = 0; k < 2; k++) {  
    double Zt = Z_arb(t_curr);          /* coût  $\approx 1.8$  ms */
```

```

    if (fabs(Zt) < tol) return t_curr; /* early-exit si convergé */
    t_curr -= Zt / dZ;
}

```

L'early-exit s'active naturellement pour $t > 50\,000$ (où le pseudo-zéro Phase 1 est suffisamment proche du vrai zéro). Mais pour la majorité des zéros (87 % ont $t > 20\,000$), le second Newton step est redondant : l'erreur après 1 step est déjà inférieure à tol.

Cause mathématique : biais $Z_{rs} \approx C \cdot t^{-5/4}$ avec $C \approx 0.305$. Ce biais détermine la distance entre le pseudo-zéro Phase 1 et le vrai zéro. À $t = 20\,000$: biais $\approx 6.4 \times 10^{-7}$, erreur 1 Newton $\approx 4 \times 10^{-13}$ — déjà inférieure à tol = 10^{-12} .

2. Solution v14 — Cache RS statique log_n / isqrt_n

2.1 Principe

v14 précompute $\log(n)$ et $1/\sqrt{n}$ pour $n = 1 \dots 2100$ et les stocke dans des tableaux statiques initialisés une fois par worker :

```

#define N_MAX_CACHE 2100 /* couvre  $T \lesssim 27M - N_{RS} = \text{floor}(\sqrt{T/2\pi})$  */

static double log_n_cache[N_MAX_CACHE + 1]; /* log(n) pour n=1..2100 */
static double isqrt_n_cache[N_MAX_CACHE + 1]; /* 1/sqrt(n) */
static int    g_cache_ready = 0;

static void init_rs_cache(void) {
    if (g_cache_ready) return; /* idempotent */
    for (int n = 1; n <= N_MAX_CACHE; n++) {
        log_n_cache[n] = log((double)n);
        isqrt_n_cache[n] = 1.0 / sqrt((double)n);
    }
    g_cache_ready = 1;
}

```

Taille mémoire : $2 \times 2101 \times 8 = 33\text{ KB}$ — tient entièrement en cache L2 (typiquement 256 KB sur i7). Aucune pression mémoire additionnelle.

Couverture : $N_{RS} = \lfloor \sqrt{T/2\pi} \rfloor \leq 2100 \Leftrightarrow T \lesssim 27\text{ M}$.

Initialisation post-fork : appelée au premier accès du worker. Dans l'architecture multiprocessing Python (fork), chaque worker a sa propre copie du cache — pas de race condition.

2.2 Boucle RS avec cache

```

/* v14+ — lecture cache : ~4 cycles/terme au lieu de ~100 cycles */
if (N <= N_MAX_CACHE) {
    for (n = 1; n <= N; n++)

```

```

        sum += cos(th - t * log_n_cache[n]) * isqrt_n_cache[n];
    } else {
        /* fallback si T > 27M (non atteint actuellement) */
        for (n = 1; n <= N; n++)
            sum += cos(th - t * log((double)n)) / sqrt((double)n);
    }

```

Économie par terme : 100 cycles (log+sqrt) → 4 cycles (lecture L2) = **×25 par terme**.

Impact global Phase 1 : réduction du coût de 3 313 656 évaluations Z_{rs_double} .

2.3 Fichiers modifiés

- src/calculs/optimisation/c_modules/illinois_arb.c — cache + boucle RS
- src/calculs/optimisation/c_modules/scan_arb.c — idem pour le scan
- src/calculs/optimisation/compute_zeros_v14.py — orchestrateur v14

3. Solution v15 — Phase 2 adaptative SEUIL_1NEWTON

3.1 Analyse mathématique du biais Z_{rs}

La formule RS tronquée à $C_0 + C_1$ en double précision produit un biais :

$$\text{biais}(t) \approx C \cdot t^{-5/4}, \quad C \approx 0.305 \quad (\text{calibré LMFDB 04/07/2026})$$

Ce biais est la distance entre le pseudo-zéro $\tilde{t}$ (obtenu par Phase 1) et le vrai zéro t_0 :

$$|\tilde{t} - t_0| \approx \frac{\text{biais}(t)}{|Z'(t_0)|}$$

Après 1 pas de Newton depuis $\tilde{t}$:

$$|x_1 - t_0| \approx \frac{(x_0 - t_0)^2 \cdot Z''(t_0)}{2 \cdot Z'(t_0)} \approx C^2 \cdot t^{-5/2} \approx 0.093 \cdot t^{-5/2}$$

3.2 Tableau biais et erreur Newton

t	$\text{biais}(t)$	$\varepsilon_1 \text{ Newton}$	$< \text{tol} = 10^{-12} ?$
65	5.0×10^{-3}	$\sim 1.75 \times 10^{-6}$	□ Non
1 000	6.6×10^{-5}	$\sim 3.1 \times 10^{-9}$	□ Non
10 000	3.7×10^{-6}	$\sim 9.8 \times 10^{-12}$	□ Limite
16 000	2.0×10^{-6}	$\sim 2.8 \times 10^{-12}$	□ Limite
20 000	6.4×10^{-7}	$\sim 4 \times 10^{-13}$	□ Oui (×2.4 marge)

t	$\text{biais}(t)$	$\varepsilon_1 \text{ Newton}$	$< \text{tol} = 10^{-12} ?$
50 000	5.4×10^{-8}	$\sim 2.9 \times 10^{-15}$	□
100 000	9.6×10^{-9}	$\sim 9.2 \times 10^{-17}$	□

Seuil retenu : SEUIL_1NEWTON = 20 000 (marge $\times 2.4$ sur le minimum strict ≈ 16 000).

3.3 Implémentation C

```
/* Dans illinois_arb.c – Phase 2 (v15) */
#define SEUIL_1NEWTON 20000.0

/* Point de départ : meilleure borne du bracket serré */
double t_curr = (fabs(Za) < fabs(Zb)) ? a : b;
double h      = 1e-4;
int n_newton = (t_curr < SEUIL_1NEWTON) ? 2 : 1;

for (int k = 0; k < n_newton; k++) {
    /* Dérivée Z'(t) via Z_rs_double (différence centrale – cache actif) */
    double dZ = (Z_rs_double(t_curr + h) - Z_rs_double(t_curr - h)) / (2.0 * h);
    if (fabs(dZ) < 1e-10) break;
    double Zt = Z_arb(t_curr); /* appel Arb – ~1.8 ms */
    if (fabs(Zt) < tol) return t_curr;
    double delta = Zt / dZ;
    t_curr -= delta;
    if (fabs(delta) < tol) break;
}
```

3.4 Distribution des zéros T=100 000 par rapport au seuil

Sur 138 069 zéros dans $[14, 100\,000]$:

Tranche t	Zéros	Part	Newton steps	Appels Z_arb
$t < 65$ (mpmath_petit_t)	~87	~0.06 %	—	0 (mpmath)
$65 \leq t < 20\,000$	~17 940	~13 %	2	4
$t \geq 20\,000$	~120 042	~87 %	1	2

Économie v15 vs v14 : 120 042 zéros $\times$ 2 appels Z_arb économisés $\times$ 1.8 ms = **432 s $\approx$ 7.2 min.**

3.5 Piège confirmé : 1 Newton fixe pour tout t

La tentative naïve de réduire à 1 Newton pour **tout** t produit des erreurs importantes pour $t < 200$:

t	biais_RS	dist. pseudo-zéro	erreur 1 Newton	vs LMFDB
14.13 (γ_1)	$\sim 5 \times 10^{-3}$	5×10^{-3}	$\sim 1.5 \times 10^{-5}$	□
21.02 (γ_2)	$\sim 3 \times 10^{-3}$	3×10^{-3}	$\sim 6 \times 10^{-6}$	□
65.11	$\sim 2 \times 10^{-3}$	2×10^{-3}	$\sim 1.75 \times 10^{-6}$	□

Résultat mesuré : LMFDB 14/20 avec 1 Newton fixe (vs 20/20 avec SEUIL_1NEWTON). Erreurs de 10^{-6} à 1.75×10^{-6} pour $t \approx 65-77$ (zéros #9-#20).

Règle impérative : toujours utiliser $n_newton = (t < SEUIL_1NEWTON) ? 2 : 1$;

4. Résultats T=100 000 v13→v15 (mesurés 2026-07-04)

4.1 Benchmark T=100 000 (PC1, 8 workers, mode turbo)

Version	Durée	Vitesse	Gain vs v13	Zéros	Turing	LMFDB
v13	8.50 min	271 z/s	—	138 069	□ COM- PLET	20/20
v14	7.7 min	299 z/s	×1.10	138 069	□ COM- PLET	20/20
v15	4.4 min	517 z/s	×1.93	138 069	□ COM- PLET	20/20

4.2 Tableau récapitulatif Avant/Après/Gain

Métrique	v13	v14	v15	Gain v13→v15
Backend affinage	Illinois hybride Z_rs + 2 Newton Arb	+ cache log_n/isqrt_n	+ SEUIL_1NEW- TON=20k	—
Coût Z_rs_double	~100 cycles/terme	~4 cycles/terme (L2)	idem v14	×25 par terme
Newton steps ($t \geq 20k$)	2	2	1	−1 appel Z_arb
Appels Z_arb ($t \geq 20k$)	4	4	2	×2
Durée T=100k	8.50 min	7.7 min	4.4 min	×1.93
Vitesse	271 z/s	299 z/s	517 z/s	×1.91
Zéros	0 □	0 □	0 □	=
manquants LMFDB	20/20 □	20/20 □	20/20 □	=

Métrique	v13	v14	v15	Gain v13→v15
Turing-Backlund	COMPLET $\square$	COMPLET $\square$	COMPLET $\square$	=

4.3 Gain cumulé v1 → v15

$$\frac{21 \text{ h}}{4.4 \text{ min}} = \frac{1\,260 \text{ min}}{4.4 \text{ min}} \approx \times 28\,600$$

Condition Objectif 2 atteinte le 04/07/2026 : T=100k = 4.4 min < 5 min $\square$

4.4 Commits

Version	Commit	Branche
v13	77efd10	Riemann_Lab_C
v14	d4b3611	Riemann_Lab_C
v15	adf5d2a	Riemann_Lab_C

5. Run T=5 000 000 — v13 (terminé 2026-07-04)

Le run T=5M lancé le 27/06/2026 à 16h02 s'est terminé le 04/07/2026 après ~38h.

Indicateur	Valeur
T_MAX	5 000 000
Zéros attendus N(5M)	10 016 473
Zéros trouvés	10 016 377
Manquants	96
Turing-Backlund	$\square$ INCOMPLET (96 manquants)
Version	v13 PC1 local
Workers	8 · turbo
STEP	0.001571 (adaptatif, MARGE=2.0)

5.1 Cause des 96 manquants

Les 96 manquants sont des **paires de zéros très proches** dont le changement de signe de $Z(t)$ ne passe pas à travers la grille d'échantillonnage (phase de grille défavorable).

Mécanisme : si deux zéros consécutifs ρ_n, ρ_{n+1} sont à distance $\delta_{n,n+1} < \text{STEP}$, et si la phase de la grille tombe entre eux, aucun changement de signe n'est détecté → zéro manqué.

Ce n'est pas un bug Illinois : les REJECT et FALLBACK sont nuls (confirmé par l'instrumentation ZETA_DEBUG_BRACKETS du 26/06/2026). L'affinage Illinois est correct pour tout bracket fourni.

Piste v15 : le cache RS et la Phase 2 adaptative n'impactent pas la détection. Le run v15 à T=5M réduira peut-être marginalement les manquants (STEP identique) ou permettra d'investiguer avec un STEP réduit dans le temps économisé.

6. Questions ouvertes

- **Run T=5M avec v15 :** économie de ~20h (35h→~18h estimé). Permettra d'investiguer un STEP plus serré pour les 96 manquants.
 - **Investigation 96 manquants :** instrumenter le scan pour logger les gaps $\delta_{n,n+1}$ mesurés vs STEP à T=5M — confirmer la corrélation gap/manquant.
 - **v16 — Odlyzko-Schönhage :** $O(N \log^2 N)$ vs $O(N\sqrt{T})$ actuel. Gain ×50+ à T=5M. Implémentation complexe (FFT non-uniforme). Nécessaire pour T > 10M.
 - **Cache RS couverture T > 27M :** actuellement N_MAX_CACHE=2100. Pour T=50M, $N_{RS} \approx 2823$ — fallback sans cache. Option : realloc dynamique ou augmenter N_MAX_CACHE à 3000 (mémoire : 48 KB, toujours en L2).
 - **Objectif 2 — agent IA :** condition numérique atteinte. Prochain jalon : Anthropic Skilljar + MCP + RAG vault (/mnt/vault_rag, SSD Micron 1100 256 Go).
-

analyse_problemes_v13_v15.md · Riemann_Lab.wiki/master · hprzeta · MAJ 2026-07-04 · ~170 lignes